

Arc Node Integration Guide

This document explains how ENTARC is integrated with Arc Network's open-source node implementation.

Source Repository: [Circle Finance Arc Node](https://github.com/circlefin/arc-node) (<https://github.com/circlefin/arc-node>)

Integration Overview

ENTARC integrates the following components from Arc Node:

1. Smart Contracts & ABIs

Location: `lib/contracts/`

- **USDC Token ABI** (`usdc-abi.ts`)
 - ERC20-compliant token interface
 - Used for balance queries and transfers
 - Deployed on Arc Testnet (Chain ID: 5042002)
- **Protocol Configuration ABI** (`protocol-config-abi.ts`)
 - Protocol fee parameters
 - Gas limit configuration
 - Base fee calculation parameters (alpha, kRate, etc.)
- **Validator Manager ABI** (`validator-manager-abi.ts`)
 - Validator registration and management
 - For future DAO and consensus features

2. Network Configuration

Location: `lib/contracts/arc-testnet-addresses.ts` and `arc-network-config.json`

Key Parameters:

- Chain ID: **5042002**
- RPC: <https://rpc.testnet.arc.network>
- Explorer: <https://testnet.arcscan.app>
- Native Currency: **ARC** (18 decimals)
- Token: **USDC** (6 decimals)

Gas Configuration:

- Block Gas Limit: 30,000,000
- Min Base Fee: 1
- Max Base Fee: 1,000
- Fee Model: Dynamic (alpha=20, kRate=25)

3. Wagmi Configuration

Location: `lib/wagmi-config.ts`

Updated to use Arc Network specifications:

- Connects to Arc Testnet RPC
- Supports MetaMask and injected wallets
- Configured with Arc's native currency (ARC, not USDC)
- Centralized contract address management

Component Usage

Reading Token Balance

```
import { useReadContract } from 'wagmi';
import { USDC_ABI, ARC_TESTNET_ADDRESSES } from '@lib/contracts';

function TokenBalance({ address }) {
  const { data: balance } = useReadContract({
    abi: USDC_ABI,
    address: ARC_TESTNET_ADDRESSES.TOKEN.ADDRESS,
    functionName: 'balanceOf',
    args: [address],
  });
  return <div>{balance?.toString()}</div>;
}
```

Checking Protocol Configuration

```
import { useReadContract } from 'wagmi';
import { PROTOCOL_CONFIG_ABI, ARC_TESTNET_ADDRESSES } from '@lib/contracts';

function ProtocolFees() {
  const { data: config } = useReadContract({
    abi: PROTOCOL_CONFIG_ABI,
    address: ARC_TESTNET_ADDRESSES.PROTOCOL_CONFIG.ADDRESS,
    functionName: 'getProtocolConfig',
  });
  return <div>Base Fee: {config?.minBaseFee}</div>;
}
```

Contract Addresses

All contract addresses are maintained in `lib/contracts/arc-testnet-addresses.ts` :

Contract	Address	Purpose
USDC Token	0x0abb...	Native stablecoin
Protocol Config	0x0000...	Fee and gas configuration
Validator Manager	0x0000...	Validator registry

Updating Integration

When Arc Node updates its testnet configuration:

1. Pull the latest from [Circle Finance Arc Node](https://github.com/circlefin/arc-node) (<https://github.com/circlefin/arc-node>)
2. Extract new contract addresses from `assets/testnet/config.json`
3. Update `lib/contracts/arc-testnet-addresses.ts`
4. Update `arc-network-config.json` with new values
5. If ABI changes, update the corresponding ABI files in `lib/contracts/`
6. Test with local Arc testnet node (if available)
7. Deploy to production

References

- [Arc Network Documentation](https://docs.arc.network) (<https://docs.arc.network>)
- [Arc Node GitHub](https://github.com/circlefin/arc-node) (<https://github.com/circlefin/arc-node>)
- [Circle Finance](https://www.circle.com) (<https://www.circle.com>)
- [Wagmi Documentation](https://wagmi.sh) (<https://wagmi.sh>)
- [Viem Documentation](https://viem.sh) (<https://viem.sh>)

Notes

- Native currency on Arc is **ARC** (18 decimals), not USDC
- USDC is the native stablecoin for transactions
- Protocol uses dynamic fee model with elastic base fee calculation
- Validator registration is permissioned (requires approved registerer address)
- All addresses and parameters are configured for **Arc Testnet** (Chain ID: 5042002)

Future Enhancements

- [] Mainnet support when Arc launches
- [] On-chain voting integration (ProtocolConfig updates)
- [] Validator stake management
- [] Advanced fee calculation UI
- [] Local testnet setup guide